

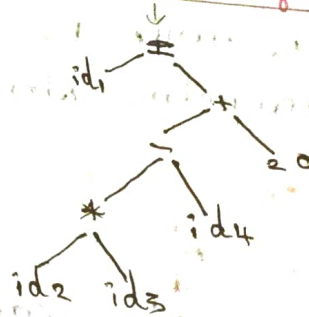
CO-1 AT:1

$$W = x * y - z + 20$$

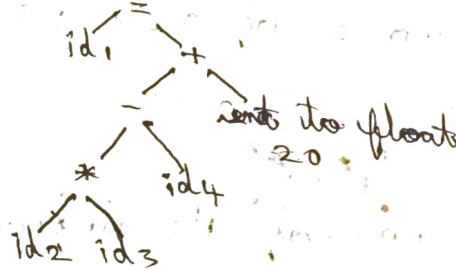
Lexical Analyzer

 $\langle id_1 \rangle \langle = \rangle \langle id_2 \rangle \langle * \rangle \langle id_3 \rangle \langle - \rangle \langle id_4 \rangle \langle + \rangle \langle 20 \rangle$

Syntax Analyzer



Semantic Analyzer



Intermediate Code Generator

 $t_1 = id_4 + \text{into the float}(20)$
 $t_2 = id_3 - t_1$
 $t_3 = id_2 * t_2$
 $id_1 = t_3$

Code Optimizer

 $t_1 = id_4 + 20.0$
 $t_1 = id_3 - t_1$
 $id_1 = id_2 * t_1$

Code Generator

LDF R1, id4

ADDF R1, R1, #20.0

LDF R2, id2

LDF R3, id3

MUL R2, R2, R3

ADD R2, R2, R1

STF id1, R2

2. i) All strings starts & end with 1:

$$1(0+1)^*1+1$$

- The string starts with 1.
- The middle part may contain any combination of 0 & 1.
- The strings ends with 1.
- The single-character string "1" is also included.

ii) All strings contains the substring 01.

$$(0+1)^*01(0+1)^*$$

- Any no. of 0's & 1's can appear before 01.
- Any substring 01 must appear at least once.
- Any no. of 0's & 1's can appear after 01.

iii) All strings with exactly one 0.

$$1^*01^*$$

- Any no. of 1's before the single 0.
- Exactly one 0 occurs.
- Any no. of 1's after the 0.

iv) All strings with atleast two 1's.

$$(0+1)^*1(0+1)^*1(0+1)^*$$

- The expression guarantee two occurrences of 1.
- Additional 0's or 1's may appear anywhere.

v) All strings that do not end with 0.

$$\epsilon + 1 + (0+1)^*1$$

- ϵ represents the empty string.
- 1 is a valid single-character string.
- All other valid strings must end with 1.
- Therefore, no string ends with 0.

i) $(\epsilon + aa^*)^*$

Simplification

$$aa^* = a^+ \text{ (one or more a's)}$$

$$\epsilon + a^+ = a^*$$

$$(a = a^*)$$

Language Generated

$$L = \{a^n \mid n \geq 0\}$$

Pattern

The language contains all string consisting only of the symbol, a including the empty string.

ii) $(a^* + b^*)^* abb$

Language Generated:

All strings over $\{a, b\}$ that end with the substring abb .

Pattern:

→ The 1st part $(a^* + b^*)^*$ generates by any combination of a & b .

→ Every valid string must end with "abb".

iii) $(a^*)^* aba$

Language Generated:

All strings over $\{a, b\}$ that contain the mandatory ending substring aba .

Pattern:

→ The expression $(a^*)^*$ generates any sequence of a 's & b 's.

→ Every valid string must end with "aba".

iv) $(a+ab)^*$

Language Generated:

Strings formed by repeatedly choosing either a or ab .

Pattern:

Each repetition may contribute,

* a

* ab

v) $(aba)^* + (a^*)^*$

Language Generated:

This is the union of 2 languages

1st part = $(aba)^*$ 2nd part = $(a^*b^*)^*$

Combined Language:

The final language contains

→ all repetition of "aba"

→ all strings over $\{a, b\}$

vi) Basic Regular Expression:

% {

include <stdio.h>

% }

letter [A-Za-z]

digit [0-9]

id { letter } ({ letter } | { digit }) *

"if"

{ printf ("Keyword : if \n"); }

"else"

{ printf ("Keyword : else \n"); }

"while"

{ printf ("Keyword : while \n"); }

{id}

```
{ printf("Identifier: %s\n", yytext); }
```

{digit} +

```
{ printf("Constant: %s\n", yytext); }
```

"+" "-" "*" "/"

```
{ printf("Operator: %s\n", yytext); }
```

"(" ")" "{" "}" ";"

```
{ printf("Delimiter: %s\n", yytext); }
```

[\t \n] ;

%;

```
int yywrap()
```

```
{ return 1;
```

```
}
```

```
int main()
```

```
{
```

```
    yylex();
```

```
    return 0;
```

```
}
```

ii) Justification:

$[A - za - z] ([A - za - z0 - 9]^*)$

→ The 1st character must be a letter.

→ The remaining characters may be letters or digits.

→ It doesn't allow an identifier to start with a digit.

→ Keywords (if, else, while) are written before the identifier rule. Hence, Lex recognizes them as keywords instead of identifiers.

Valid Identifiers:

count

21

Total

sum25

abc123

Reason: All start with a letter & contain only letters & digits.

Invalid Identifiers:

1count

25abc

@name

-a

a-b

Reason:

→ Starts with a digit

→ Contains the operator

→ Starts with -, which is not allowed by the given specification

Valid Integer Constants

0

25

100

4567

Invalid Numeric Formats:

12a

1.5

abc123

$$5. i) (0^*1^*)^* = (0+1)^*$$

LHS:

$$(0^*1^*)^*$$

→ 0^* generates zero or more 0s

→ 1^* generates zero or more 1s

→ Repeating these blocks using * generates every possible combination of 0 & 1

Hence

$$(0^*1^*)^* = (0+1)^*$$

$$\text{LHS} = \text{RHS}$$

$$ii) (0+1)^* = (0+1)^*$$

LHS

$$(0+1)^*(0+1)^*$$

→ $(0+1)^*$ already represents all binary strings.

→ Concatenating 2 languages that already contain all binary strings doesn't produce any new string.

Hence

$$(0+1)^* (0+1)^* = (0+1)^*$$

$$\text{LHS} = \text{RHS}$$

iii) $(0=0)^*$

$$(0^*)^*$$

→ 0^* generates all strings containing zero or more 0s.

→ Applying the Kleene star again doesn't change the language.

Hence:

$$(0^*)^* = 0^*$$

$$\text{LHS} = \text{RHS}$$

iv) $(0+1) + (0+1)^* = (0+1)^*$

1st term = $(0+1)^* 0 (0+1)^*$

2nd term = $(0+1)^* 1 (0+1)^*$

Generates all binary strings containing at least one 1.

Hence Union of these 2.

$$(0+1)^* 0 (0+1)^* + (0+1)^* 1 (0+1)^* = (0+1)^*$$

$$\text{LHS} = \text{RHS}$$

v) $\epsilon + 0(0)^* = 0^*$

LHS

$$\epsilon + 0(0)^*$$

→ ϵ represents the empty string

→ $0(0)^*$ represents one or more 0s

→ Together they represent zero or more 0s

$$\epsilon + 0(0)^* = 0^*$$

$$\text{LHS} = \text{RHS}$$